Group A

Assignment No. 1

Title: Design and Implementation of a Scientific Calculator using Java

Objective: To design and implement a Scientific Calculator using Java programming that performs basic arithmetic operations and scientific functions such as trigonometric, logarithmic, exponential, and power calculations.

Theory: A Scientific Calculator is an advanced calculator capable of performing complex mathematical operations beyond basic arithmetic. Java provides a built-in `Math` class, which supports various mathematical functions such as:

- `Math.sin()`, `Math.cos()`, `Math.tan()` for trigonometric operations
- `Math.log()` for logarithmic calculation
- `Math.sqrt()` for square root
- `Math.pow()` for power calculation
- `Math.exp()` for exponential functions

In this experiment, a menu-driven Java program is developed that allows the user to select the desired operation and input the required values. The program uses conditional statements, switch-case, and loops to execute operations repeatedly until the user exits.

Algorithm:

1. Start the program.
2. Display a menu of scientific calculator operations.
3. Accept the user's choice.
4. Read required input values.
5. Perform the selected operation using Java Math functions.
6. Display the result.
7. Ask the user whether to continue.

8. Stop the program.

Menu-driven programs in Java are implemented using a `do-while` loop to repeatedly display the menu and a `switch-case` statement to execute the selected option.

Sample Program Code:

```
do {
    // Display menu
    System.out.println("1. Option 1");
    System.out.println("2. Option 2");
    System.out.println("0. Exit");

    // Read user choice
    choice = sc.nextInt();

    // Execute selected option
    switch (choice) {
        case 1:
            // Statements for Option 1
            break;

        case 2:
            // Statements for Option 2
            break;

        case 0:
            // Exit statement
            break;

        default:
            System.out.println("Invalid choice");
    }
} while (choice != 0);
```

Conclusion: This experiment demonstrates the effective use of Java programming and the `Math` class to build a scientific calculator. The program enhances understanding of control structures, user input handling, and built-in mathematical functions.

Assignment No. 2

Title: Write a program to perform following array operations – addition of numbers, find min/max element, sort array, search

elements, find sum and average of array elements.

Objective:

- To understand the concept of arrays in Java.
- To perform basic array operations such as input, display, search, replace, addition, sum, average, and finding minimum and maximum elements.
- To implement array-based logic using Java syntax.

Theory:

1. **Declaring an Array** Used to create an array that can store multiple values of the same data type.

```
int[] arr = new int[size];
```

2. **Getting Data (Input Elements into Array)** Used to store values into the array using a loop.

```
for(int i = 0; i < size; i++) {
    arr[i] = value;
}
```

3. **Displaying Array Elements** Used to access and print all elements of the array.

```
for(int i = 0; i < size; i++) {
    System.out.println(arr[i]);
}
```

4. **Searching an Element in Array** Used to check whether a specific element exists in the array.

```
for(int i = 0; i < size; i++) {
    if(arr[i] == key) {
        // element found
    }
}
```

5. **Replacing an Element in Array** Used to replace an existing value with a new value.

```
for(int i = 0; i < size; i++) {
    if(arr[i] == oldValue) {
        arr[i] = newValue;
    }
}
```

6. **Addition of Array Elements (Element-wise / Total)** Used to add values stored in the array.

```
int sum = 0;
for(int i = 0; i < size; i++) {
    sum = sum + arr[i];
}
```

7. **Finding Sum and Average of Array Elements** Used to calculate total and mean of elements.

```
int sum = 0;
for(int i = 0; i < size; i++) {
    sum += arr[i];
}
double average = (double) sum / size;
```

8. **Finding Minimum Element in Array** Used to determine the smallest value in the array.

```
int min = arr[0];
for(int i = 1; i < size; i++) {
    if(arr[i] < min) {
        min = arr[i];
    }
}
```

9. **Finding Maximum Element in Array** Used to determine the largest value in the array.

```
int max = arr[0];
for(int i = 1; i < size; i++) {
    if(arr[i] > max) {
        max = arr[i];
    }
}
```

Conclusion: Thus, basic array operations such as input, display, search, replace, sum, average, minimum, and maximum were studied and implemented using Java arrays. This experiment helps in understanding data storage and manipulation using arrays in Java programming.

Assignment No. 3

Title: Write a JAVA code to invoke `getData()`, `displayData()` and `searchElement()` methods using array of objects (student, book, account etc).

Objective: To study and implement the concept of array of objects in Java by creating a class (`Student`) and invoking the methods `getData()`, `displayData()`, and `searchElement()` to store, display, and search records.

Theory: In Java, an array of objects is used when we need to store multiple objects of the same class. Each element of the array refers to an object. Methods such as `getData()` are used to accept input, `displayData()` to display stored information, and `searchElement()` to search a particular record based on a key value.

Syntax:

```
ClassName obj[] = new ClassName[size];
obj[i] = new ClassName();
void getData() { }
void displayData() { }
static void searchElement(ClassName obj[], key) { }
```

Sample Program Code:

```java
import java.util.Scanner;

class Student {
    int rollNo;
    String name;

    void getData() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Roll No: ");
        rollNo = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Name: ");
        name = sc.nextLine();
    }

    void displayData() {
        System.out.println("Roll No: " + rollNo + ",
    }

    static void searchElement(Student s[], int key) {
        boolean found = false;
        for (Student st : s) {
            if (st.rollNo == key) {
                st.displayData();
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Student not found.");
        }
    }
}

public class ArrayOfObjectsDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of students: "
        int n = sc.nextInt();
        Student s[] = new Student[n];

        for (int i = 0; i < n; i++) {
            s[i] = new Student();
            s[i].getData();
        }

        System.out.println("\nStudent Details:");
        for (Student st : s) {
            st.displayData();
        }
```

```
        System.out.print("\nEnter roll number to sear
        int key = sc.nextInt();
        Student.searchElement(s, key);
    }
}
```

Conclusion: Thus, the program successfully demonstrates the use of an array of objects in Java. The methods `getData()`, `displayData()`, and `searchElement()` were implemented to store, display, and search object data efficiently.

Assignment No. 4

Title: Program to Demonstrate the Use of Access Modifiers in Java Within and Outside the Package

Objective: To understand and implement Java access modifiers (`public`, `protected`, default, and `private`) and study their accessibility within the same package and outside the package.

Theory: Access modifiers in Java define the visibility and scope of class members such as variables and methods. Java provides four types of access modifiers:

- `private` – Accessible only within the same class
- default (no modifier) – Accessible within the same package
- `protected` – Accessible within the same package and in subclasses outside the package
- `public` – Accessible from anywhere

By declaring variables with different access modifiers in a class, we can observe their accessibility rules within the package and outside the package.

Syntax:

```
private dataType variableName;
dataType variableName; // default access
protected dataType variableName;
public dataType variableName;
package packageName;
import packageName.ClassName;
```

Program Code:

File 1: Package Definition (Within Package)

```
package p1;

public class AccessDemo {
    public int a = 10;
    protected int b = 20;
    int c = 30; // default
    private int d = 40;

    public void display() {
        System.out.println("Public: " + a);
        System.out.println("Protected: " + b);
        System.out.println("Default: " + c);
        System.out.println("Private: " + d);
    }
}
```

File 2: Same Package Access

```
package p1;

public class SamePackage {
    public static void main(String[] args) {
        AccessDemo obj = new AccessDemo();
        System.out.println(obj.a);
        System.out.println(obj.b);
        System.out.println(obj.c);
        // System.out.println(obj.d); // Not accessib
    }
}
```

File 3: Outside Package Access

```java
package p2;
import p1.AccessDemo;

public class OutsidePackage extends AccessDemo {
    public static void main(String[] args) {
        OutsidePackage obj = new OutsidePackage();
        System.out.println(obj.a);
        System.out.println(obj.b);
        // System.out.println(obj.c); // Not accessib
        // System.out.println(obj.d); // Not accessib
    }
}
```

Accessibility of Access Modifiers in Java:

Access Modifier	Same Class	Same Package	Outside Package (Subclass)	Outside Package (Non-Subclass)
`private`	✓	✗	✗	✗
default	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

Conclusion: Thus, the program successfully demonstrates the working of Java access modifiers. It shows that public members are accessible everywhere, protected members are accessible within the package and in subclasses outside the package, default members are accessible only within the same package, and private members are accessible only within the same class.

#file:java

Group B

Assignment No. 6

Title: Implement student class using following concepts: i) All types of Constructors. ii) Static variables and instance variable. iii) Static blocks and instance blocks. iv) Static methods and instance methods.

Objective: To understand the concepts of constructor, types of constructors, Static variable, Static blocks, Static methods and instance variables, instance blocks, instance methods.

Theory:

Constructor: Constructors in Java are a special type of method that can help create an object and assign values to its instance variables upon its initialization. Constructors have no return values, and they usually have the same name as the class. Initializing object member variables in constructors is not required but is advised for quick object setup.

1. **Default Constructor in Java:** Default Constructors in Java are powerful tools for initializing fields in a class and object in Java. They are normally declared without any arguments, allowing the compiler to automatically create the constructor when an instance of the class is created.
2. **No argument constructor in Java:** In Java, no argument constructors allow objects to be initialized with no data passed into them.
3. **Parameterized constructor in Java:** Parameterized constructors in Java allow developers to provide more flexibility when constructing objects. These constructors accept arguments that can be used for initializing the object's state.

Static Variables: Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block. They are created when the program starts and destroyed when the program stops.

Instance Variables: Instance variables are declared in a class, but outside a method, constructor or any block. They are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.

Static Blocks: Static initializer block or static initialization block executes only once during the class loading. The static blocks always execute first before the `main()` method in Java.

Instance Block: The purpose of the instance initializer block is to initialize the instance data members. It executes before constructors but after static blocks during object creation.

Static Methods: A static method in Java is a method that can be called without creating an object of the class.

Instance Methods: Instance methods are the methods defined under a class and we can call such functions only after creating an object of that class.

Code:

```java
class Student {
    // ii) Static variables and instance variables
    static String collegeName = "ABC Engineering Coll
    int rollNo;
    String name;

    // iii) Static block
    static {
        System.out.println("Static Block executed.");
    }

    // iii) Instance block
    {
        System.out.println("Instance Block executed."
    }

    // i) No-argument constructor
    Student() {
        System.out.println("No-Argument Constructor c
        this.rollNo = 0;
        this.name = "Unknown";
    }

    // i) Parameterized constructor
    Student(int rollNo, String name) {
        System.out.println("Parameterized Constructor
        this.rollNo = rollNo;
        this.name = name;
    }

    // iv) Static method
    static void displayCollege() {
        System.out.println("College Name: " + college
    }

    // iv) Instance method
    void displayStudent() {
        System.out.println("Roll No: " + rollNo + ",
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Inside Main Method --

        // Calling Static Method
        Student.displayCollege();

        System.out.println("\n--- Creating Student 1
        Student s1 = new Student();
```

```
        s1.displayStudent();

        System.out.println("\n--- Creating Student 2
        Student s2 = new Student(101, "Alice");
        s2.displayStudent();
    }
}
```

Output:

```
Static Block executed.
--- Inside Main Method ---
College Name: ABC Engineering College

--- Creating Student 1 ---
Instance Block executed.
No-Argument Constructor called.
Roll No: 0, Name: Unknown

--- Creating Student 2 ---
Instance Block executed.
Parameterized Constructor called.
Roll No: 101, Name: Alice
```

Conclusion: In this way we have studied and implemented the concepts of constructor, types of constructors, Static variable, Static blocks, Static methods and instance variables, instance blocks, instance methods.

Assignment No. 7

Title: Create a class with overloaded methods for performing mathematical operations (addition, subtraction, multiplication and division) for both integers and doubles. Demonstrate these overloaded methods with different data types.

Objective: To understand the concepts of method overloading and implementing method overloading.

Theory: If a class has multiple methods having same name but different in parameters, it is known as Method Overloading. If we have to perform only one operation, having same name of the

methods increases the readability of the program. There are two ways to overload the method in Java:

1. By changing number of arguments
2. By changing the data type

Code:

```
class MathOperations {
    // Overloaded methods for integers
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c;

    int subtract(int a, int b) { return a - b; }

    int multiply(int a, int b) { return a * b; }
    int multiply(int a, int b, int c) { return a * b

    int division(int a, int b) {
        if (b == 0) throw new ArithmeticException("Di
        return a / b;
    }

    // Overloaded methods for doubles
    double add(double a, double b) { return a + b; }

    double subtract(double a, double b) { return a -

    double multiply(double a, double b) { return a *

    double division(double a, double b) {
        if (b == 0.0) throw new ArithmeticException("
        return a / b;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();

        System.out.println("--- Integer Operations --
        System.out.println("Addition (2 params): " +
        System.out.println("Addition (3 params): " +
        System.out.println("Subtraction: " + math.sub
        System.out.println("Multiplication (2 params)
        System.out.println("Division: " + math.divisi

        System.out.println("\n--- Double Operations -
        System.out.println("Addition: " + math.add(10
        System.out.println("Subtraction: " + math.sub
        System.out.println("Multiplication: " + math.
        System.out.println("Division: " + math.divisi
    }
}
```

Output:

```
--- Integer Operations ---
Addition (2 params): 15
Addition (3 params): 17
Subtraction: 5
Multiplication (2 params): 50
Division: 2

--- Double Operations ---
Addition: 15.7
Subtraction: 5.3
Multiplication: 54.6
Division: 2.019230769230769
```

Conclusion: In this way we understood the concept of method overloading and executed programs on it.

Assignment No. 8

Title: Write a java program to calculate the area of triangle, square and circle using function overloading. Function parameters accept from user. (Use function Overloading concepts and inheritance)

Objective: To understand the concepts of method overloading, inheritance implementing method overloading and inheritance.

Theory: Inheritance is the process by which one class (the child or subclass) inherits the properties and behaviors of another class (the parent or superclass). Polymorphism using inheritance can be achieved through method overriding and method overloading. Method Overloading: When a class has multiple methods with the same name but different parameters, it is known as method overloading. Using scanner once can accept the input from the user.

Code:

```
import java.util.Scanner;

class Shape {
    // Area of Square
    double calculateArea(double side) {
        return side * side;
    }

    // Area of Circle
    double calculateArea(double radius, boolean isCir
        return Math.PI * radius * radius;
    }

    // Area of Triangle
    double calculateArea(double base, double height)
        return 0.5 * base * height;
    }
}

public class Main extends Shape {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Main shapeCalc = new Main(); // Using inherit

        System.out.println("--- Area Calculator ---")

        System.out.print("Enter side of square: ");
        double side = sc.nextDouble();
        System.out.println("Area of Square: " + shape

        System.out.print("\nEnter radius of circle: "
        double radius = sc.nextDouble();
        System.out.println("Area of Circle: " + shape

        System.out.print("\nEnter base of triangle: "
        double base = sc.nextDouble();
        System.out.print("Enter height of triangle: "
        double height = sc.nextDouble();
        System.out.println("Area of Triangle: " + sha

        sc.close();
    }
}
```

Output:

```
--- Area Calculator ---
Enter side of square: 5
Area of Square: 25.0

Enter radius of circle: 7
Area of Circle: 153.93804002589985

Enter base of triangle: 10
Enter height of triangle: 12
Area of Triangle: 60.0
```

Conclusion: In this manner, we have implemented the function overloading using inheritance.

Assignment No. 9

Title: Design an abstract class "Shape" with abstract methods for calculating area and perimeter. Create subclasses for specific shapes (eg: circle, rectangle, triangle) that extend the abstract class and implement these methods.

Objective: To understand the concepts of abstract method and abstract class and implementing the same.

Theory: Data abstraction is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with either abstract classes or interfaces. The abstract keyword is a non-access modifier, used for classes and methods:

- **Abstract class:** A restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** Can only be used in an abstract class, and it does not have a body. The body is provided by the subclass.

Code:

```java
abstract class Shape {
    abstract double calculateArea();
    abstract double calculatePerimeter();
}

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    double calculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}

class Rectangle extends Shape {
    double length, width;

    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    double calculateArea() {
        return length * width;
    }

    @Override
    double calculatePerimeter() {
        return 2 * (length + width);
    }
}

class Triangle extends Shape {
    double sideA, sideB, sideC;

    Triangle(double sideA, double sideB, double sideC
        this.sideA = sideA;
        this.sideB = sideB;
        this.sideC = sideC;
    }
```

```
    @Override
    double calculateArea() {
        // Heron's formula
        double s = (sideA + sideB + sideC) / 2;
        return Math.sqrt(s * (s - sideA) * (s - sideB
    }

    @Override
    double calculatePerimeter() {
        return sideA + sideB + sideC;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(4, 6);
        Shape triangle = new Triangle(3, 4, 5);

        System.out.println("Circle -> Area: " + circl
        System.out.println("Rectangle -> Area: " + re
        System.out.println("Triangle -> Area: " + tri
    }
}
```

Output:

```
Circle -> Area: 78.53981633974483, Perimeter: 31.4159265
Rectangle -> Area: 24.0, Perimeter: 20.0
Triangle -> Area: 6.0, Perimeter: 12.0
```

Assignment No. 10

Title: Define an interface "Employee" with methods for calculating salary and displaying employee information. Create classes that implement this interface for different types of employees (eg Fulltime, parttime).

Objective: To understand the concepts of Interface and implement the same.

Theory: A Java interface is like a blueprint of a class. It is an abstract type that specifies the behaviour of a class. Java interfaces contain static constants and abstract methods. In simple words, we can say

that an interface program in Java can have variables and abstract methods, but it can't have a method body. A few key points to remember about an interface in Java:

- An interface mentions what a class must do rather than how.
- It represents the Is-A relationship.
- It doesn't have a constructor.
- Methods declared in an interface are abstract by default.

Code:

```
interface Employee {
    double calculateSalary();
    void displayInformation();
}

class FullTimeEmployee implements Employee {
    String name;
    double monthlySalary;

    FullTimeEmployee(String name, double monthlySalar
        this.name = name;
        this.monthlySalary = monthlySalary;
    }

    @Override
    public double calculateSalary() {
        return monthlySalary;
    }

    @Override
    public void displayInformation() {
        System.out.println("Full-Time Employee: " + n
        System.out.println("Monthly Salary: $" + calc
    }
}

class PartTimeEmployee implements Employee {
    String name;
    double hourlyRate;
    int hoursWorked;

    PartTimeEmployee(String name, double hourlyRate,
        this.name = name;
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    @Override
    public double calculateSalary() {
        return hourlyRate * hoursWorked;
    }

    @Override
    public void displayInformation() {
        System.out.println("Part-Time Employee: " + n
        System.out.println("Total Salary: $" + calcul
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Employee ft = new FullTimeEmployee("Alice", 5
        Employee pt = new PartTimeEmployee("Bob", 20,

        ft.displayInformation();
        System.out.println();
        pt.displayInformation();
    }
}
```

Output:

```
Full-Time Employee: Alice
Monthly Salary: $5000.0

Part-Time Employee: Bob
Total Salary: $2000.0
```

Conclusion: In this way, we have studied the interface concept and implemented it.

Assignment No. 11

Title: Demonstrating the Use of final Keyword in Java with an Example

Objective:

1. To understand the usage of the final keyword in Java.
2. To demonstrate how the final keyword can be applied to variables, methods, and classes.
3. To explore the effect of using final on variable values, method overriding, and class inheritance.

Theory: The final keyword in Java is used to apply restrictions. It can be used in three contexts:

1. **Final Variables:** A variable declared as final cannot have its value changed once initialized. This makes the variable a constant.
2. **Final Methods:** A method declared as final cannot be overridden by subclasses. This is useful when you want to

ensure that a specific method implementation is used throughout the inheritance hierarchy.

3. **Final Classes:** A class declared as final cannot be subclassed. This is useful when you want to prevent inheritance of a class.

Code:

1. Final Variable Example

```
class DemoFinal {
    // Final variable: Cannot be modified
    final int constantValue = 10;

    public void displayMessage() {
        // constantValue = 200; // This will cause a
        System.out.println("This is a final variable
    }
}

public class Main1 {
    public static void main(String[] args) {
        DemoFinal demo = new DemoFinal();
        System.out.println("Constant Value: " + demo.
        demo.displayMessage();
    }
}
```

Output 1:

```
Constant Value: 10
This is a final variable example.
```

2. Final Method Example

```java
class DemoFinalMethod {
    public final void displayMessage() {
        System.out.println("This is a final method.")
    }
}

class SubDemoFinal extends DemoFinalMethod {
    // public void displayMessage() {
    //     System.out.println("This is a final method
    // }
}

public class Main2 {
    public static void main(String[] args) {
        SubDemoFinal demo = new SubDemoFinal();
        demo.displayMessage();
    }
}
```

Output 2:

```
This is a final method.
```

3. Final Class Example

```java
final class DemoFinalClass {
    public void displayMessage() {
        System.out.println("This is a method inside a
    }
}

// class Subclass extends DemoFinalClass {
//     // This will cause a compilation error because
// }

public class Main3 {
    public static void main(String[] args) {
        DemoFinalClass demo = new DemoFinalClass();
        demo.displayMessage();
    }
}
```

Output 3:

```
This is a method inside a final class.
```

Conclusion: The final keyword in Java ensures immutability, prevents method overriding, and avoids inheritance, promoting more reliable and secure code. It enforces constraints at the compilation level, making the program predictable and easier to maintain.